Лабораторная работа №2

Сингулярное разложение изображений, степенной метод и метод вращений Якоби

Дисциплина: Численные методы анализа

Выполнили: Колпакова Дарья Сергеевна, Недозорова Алиса Алексеевна

Группа: J3112

Санкт-Петербург, 2026

1. Цель работы

Исследовать вычисление сингулярного разложения матриц на примере обработки изображений, сравнить эффективность степенного метода и метода вращений Якоби, изучить применение усеченного SVD для сжатия данных и оценить влияние числа итераций на точность восстановления.

---

2. Краткие теоретические сведения

2.1 Что такое SVD и зачем оно нужно для изображений

Сингулярное разложение матрицы:

$$A = U\Sigma V^T,$$

где:

- U — матрица левых сингулярных векторов,
- V — матрица правых сингулярных векторов,
- Σ — диагональная матрица сингулярных чисел.

Для изображения (или отдельного цветового канала) это удобно тем, что главная структура изображения сосредоточена в старших сингулярных числах, а слабые детали и шум чаще попадают в младшие компоненты.

Сингулярные числа связаны с собственными значениями матрицы A^TA:

$$\sigma_i = \sqrt{\lambda_i(A^TA)}.$$

2.2 Степенной метод (частичная проблема собственных значений)

Считаем, что собственные значения матрицы действительны и упорядочены по модулю:

$$|\lambda_1| > |\lambda_2| > \ldots > |\lambda_r| > 0,$$

где (r) — ранг матрицы. Тогда степенной метод задается как

$$x_{k+1} = \frac{Ax_k}{\|Ax_k\|}, \quad x_0 \neq 0.$$

Идея метода: многократное умножение на матрицу усиливает компоненту, соответствующую собственному вектору с максимальным по модулю собственным

значением.
Приближение к собственному значению берется по отношению Рэлея:

$$\rho(x_k) = \frac{(Ax_k, x_k)}{(x_k, x_k)}.$$

Почему метод сходится: если для матрицы выполняется условие

$$|\lambda_1| > |\lambda_2| > \ldots > |\lambda_r| > 0$$

и начальный вектор имеет ненулевую проекцию на собственный вектор $u^{(1)}$, то при итерациях вклад компоненты, связанной с λ_1, доминирует, а остальные компоненты относительно затухают как степени отношений $|\lambda_j/\lambda_1|^k$.

Для получения нескольких старших компонент используется дефляция:

$$A_2 = A - \lambda_1 u^{(1)}(u^{(1)})^T.$$

Как дефляция используется на практике: после нахождения первой компоненты $(\lambda_1, u^{(1)})$ из рабочей матрицы удаляется ее вклад. Затем степенной метод запускается уже на новой матрице A_2, где доминирует следующая по величине компонента. Повторяя этот шаг, последовательно получаем набор старших собственных значений и векторов, а значит и старшие сингулярные числа.

2.3 Метод вращений Якоби (полная проблема собственных значений)

Применяем метод плоских вращений Якоби для симметричной матрицы с действительными элементами. Используется итерационный процесс:

$$A_{k+1} = S_k^T A_k S_k.$$

На каждом шаге выбирается пара индексов (p, q), и ортогональная матрица вращения S_k зануляет пару симметричных внедиагональных элементов: a_{pq} и a_{qp}.
Параметры вращения:

$$t = \frac{2a_{pq}}{a_{pp} - a_{qq}},$$

$$\cos\varphi = \sqrt{\frac{1}{2}\left(1 + \frac{1}{\sqrt{1+t^2}}\right)}, \quad \sin\varphi = \operatorname{sign}(t)\sqrt{\frac{1}{2}\left(1 - \frac{1}{\sqrt{1+t^2}}\right)}.$$

Повторяя шаги, матрицу приводят к почти диагональному виду, а диагональные элементы дают собственные значения.

Приближение к матрице собственных векторов накапливается по правилу:

$$U_{k+1} = U_k S_k, \quad U_0 = E.$$

2.4 Усеченное SVD

Для сжатия используется приближение ранга k:

$$A_k = U_k \Sigma_k V_k^T, \quad k < \min(n, m).$$

Чем меньше k, тем сильнее сжатие, но больше потери качества.

---

3. Ход работы

3.1 Подготовка изображения

- Загружено изображение формата `.jpg` ;
- изображение приведено к размеру 640×640;
- получены матрицы каналов R, G, B.

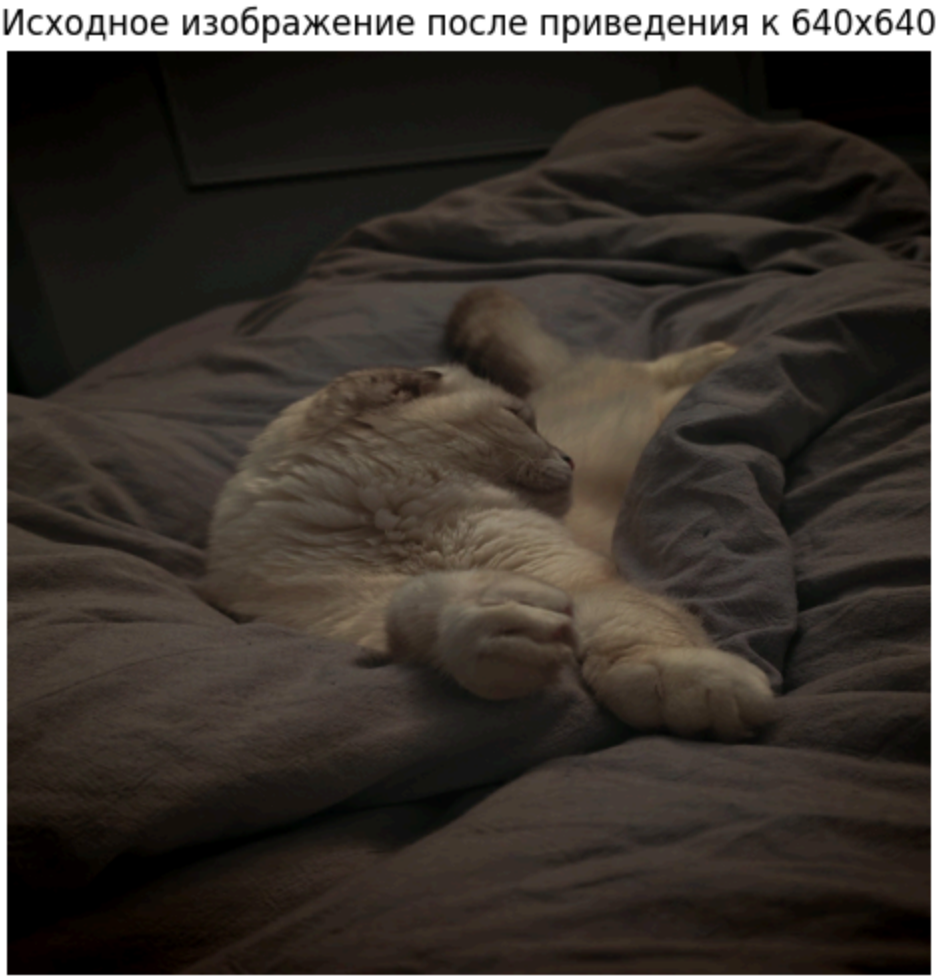


Рисунок 1 — Исходное изображение, приведенное к рабочему размеру 640x640.

Исходное изображение имеет нейтральный характер и удовлетворяет требованиям задания, на изображении показана милая кошка.



Рисунок 2 — Матрицы цветовых каналов R, G, B.

3.2 Расчет SVD степенным методом

Для каждого канала рассчитано разложение через собственные значения $A^T A$ при:

$$N \in \{10, 50, 100\}.$$

Использовались метрики:

- абсолютная норма Фробениуса:

$$\|A - \hat{A}\|_F;$$

- относительная ошибка:

$$\frac{\|A - \hat{A}\|_F}{\|A\|_F}.$$

В код добавлена проверка отрицательных оценок собственных значений:

- если $\lambda < 0$, выводится предупреждение,
- затем значение обнуляется перед извлечением квадратного корня.

Это нужно для пункта задания 2.c и корректной интерпретации численных артефактов.

	Канал	N	Absolute Frobenius error	Число найденных сингулярных чисел	Относительная ошибка восстановления	Время, сек
0	R	10	3063.271730	34	0.081574	2.575318
1	G	10	2237.426595	40	0.068508	1.486731
2	B	10	2100.453891	40	0.074974	1.543242
3	R	50	2342.906881	40	0.062391	2.572057
4	G	50	2174.705142	40	0.066588	1.876225
5	B	50	2064.103573	40	0.073677	1.953805
6	R	100	2342.557731	40	0.062381	2.442276
7	G	100	2174.589021	40	0.066584	2.296026
8	B	100	2063.354137	40	0.073650	2.310592

Таблица 1 — Результаты для $N = 10, 50, 100$: число найденных компонент, абсолютная/относительная ошибка и время.

Графики распределения сингулярных чисел по каналам изображения.

Ниже добавлены отдельные графики распределения сингулярных чисел по каждому каналу:

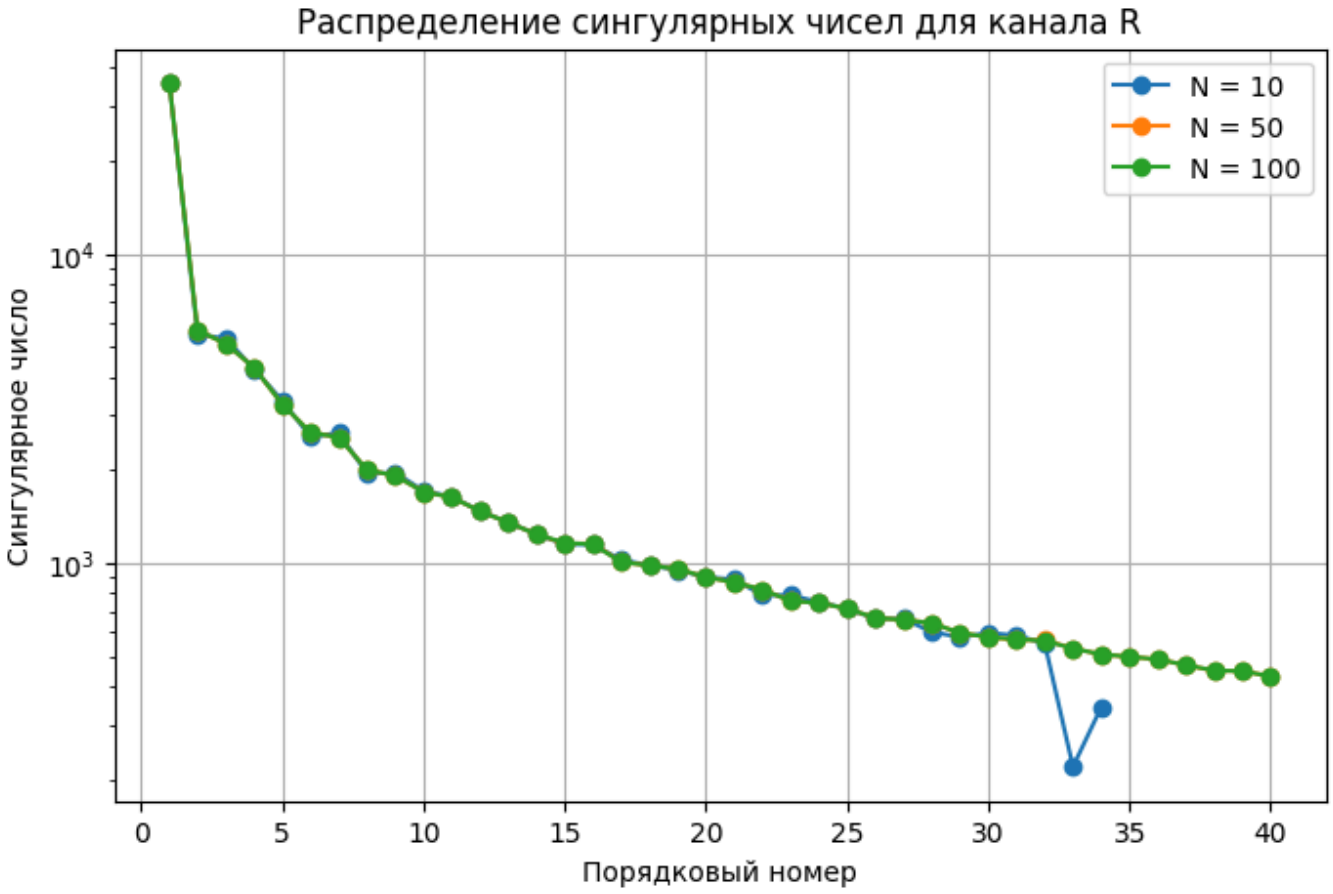


Рисунок 3.1 — Сингулярные числа канала R.

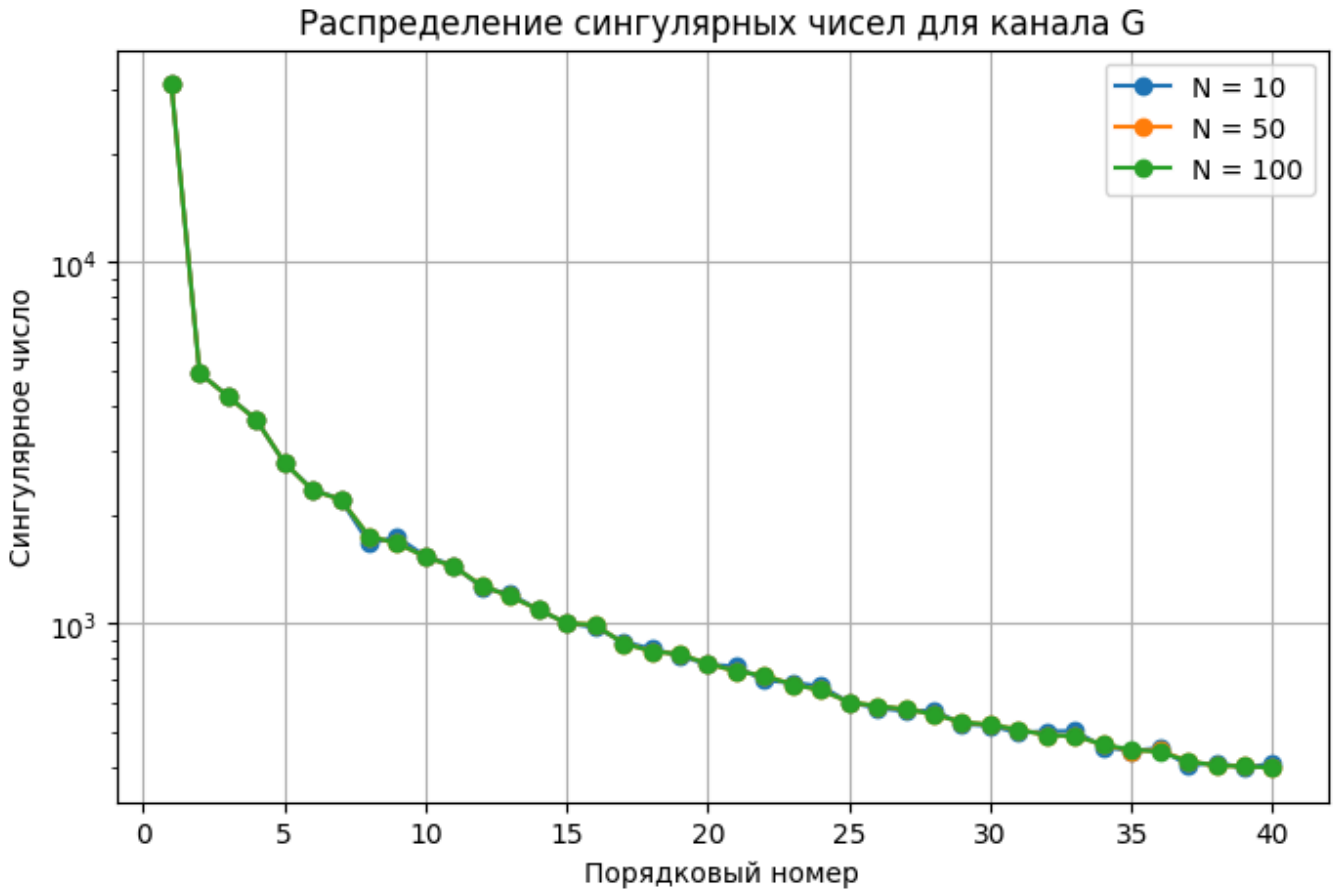


Рисунок 3.2 — Сингулярные числа канала G.

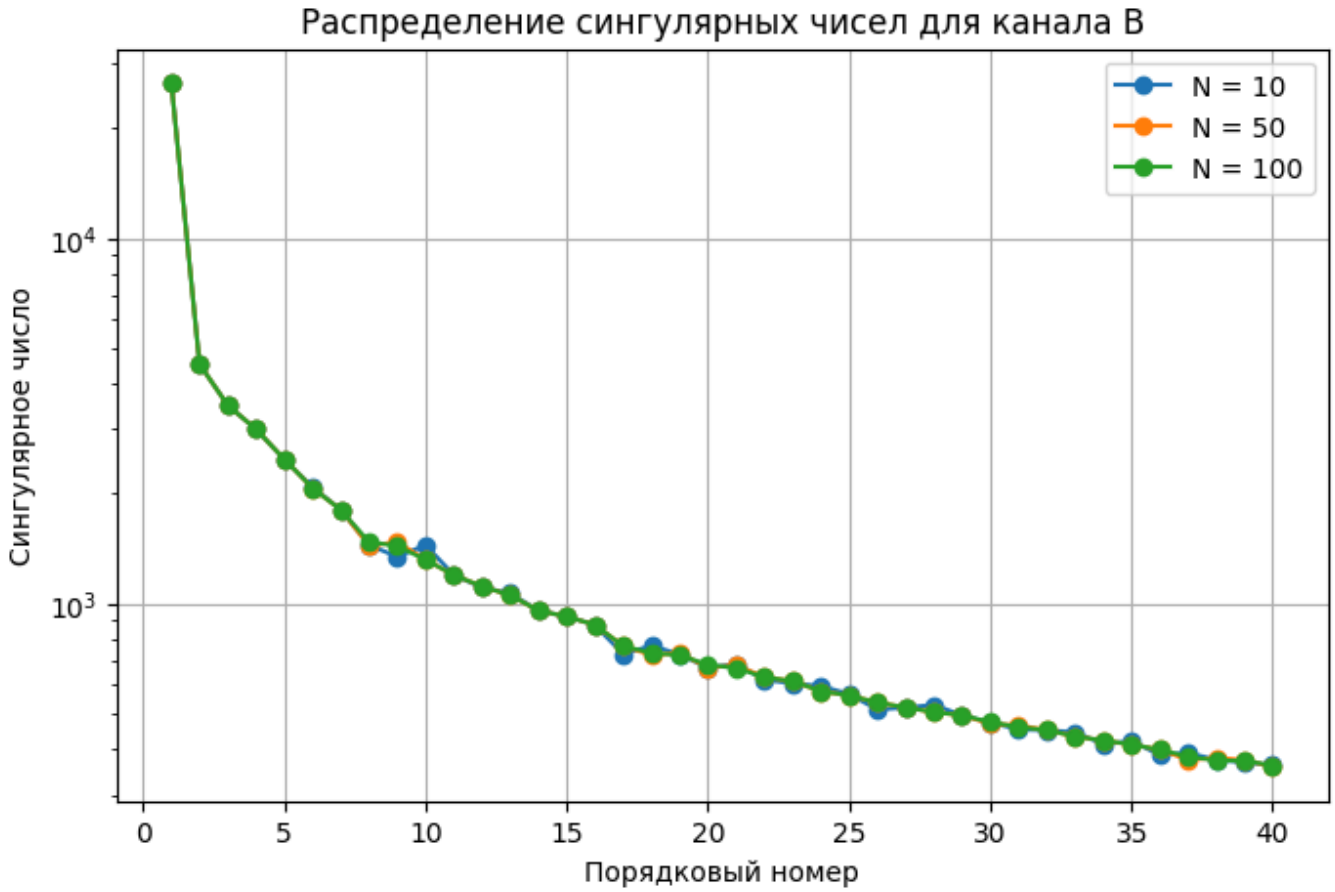


Рисунок 3.3 — Сингулярные числа канала B.

3.3 Восстановление изображения усеченным SVD

Рассмотрены значения:

$$k = \{5, 10, 20, 40\}.$$

Для каждого k рассчитаны:

- абсолютная ошибка,
- относительная ошибка,
- степень сжатия.



Рисунок 4 — Восстановленные изображения для $k = \{5, 10, 20, 40\}$.

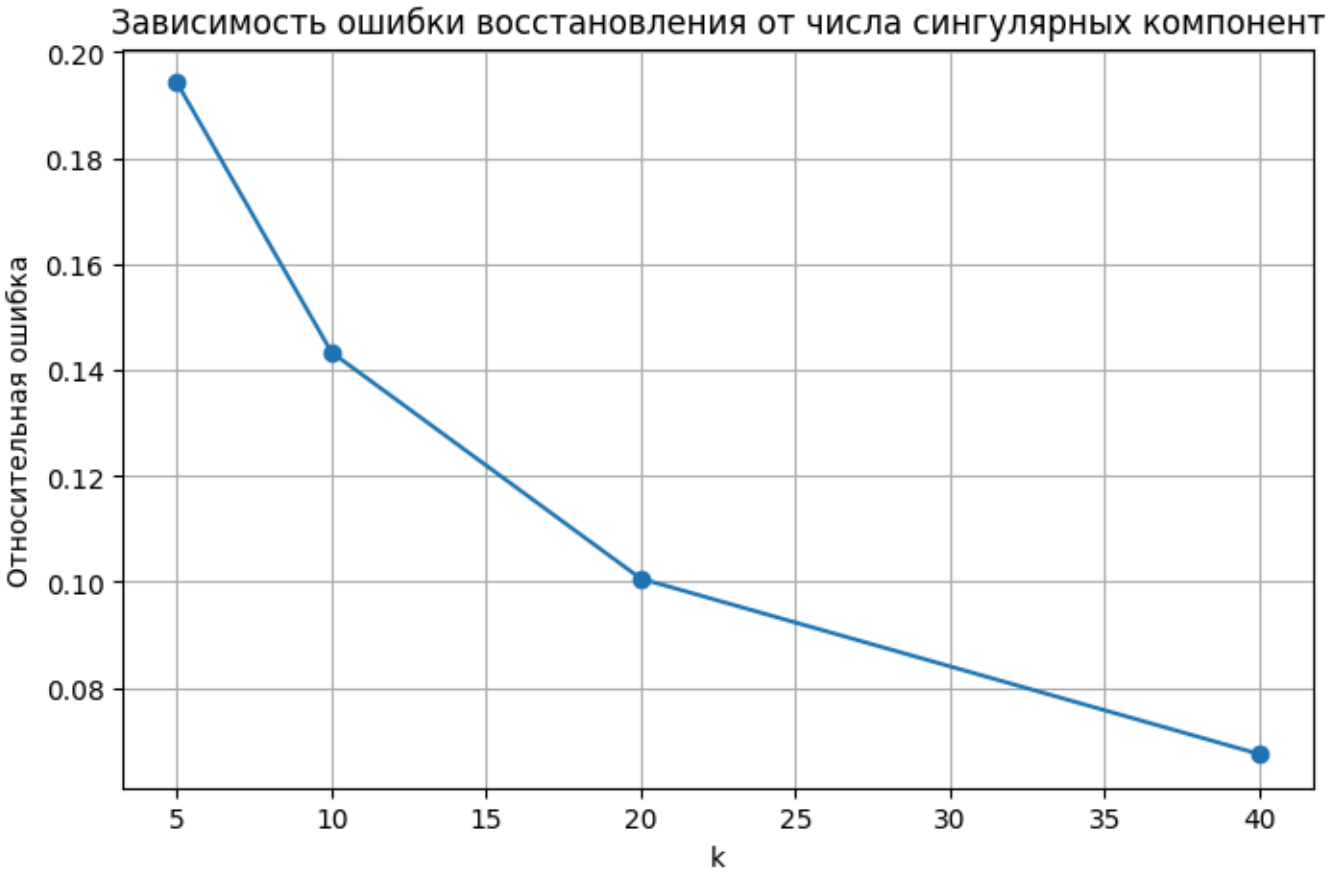


Рисунок 5 — Зависимость ошибки восстановления от числа сохраненных сингулярных чисел k.

3.4 Метод Якоби для одного канала

Для метода Якоби выбрана уменьшенная матрица 64×64 одного цветового канала.

Пояснение: метод Якоби в прямой реализации вычислительно тяжелый; на размере 640×640 время расчета в Python было бы слишком большим для практического выполнения лабораторной работы.

3.5 Сравнение методов по операциям

В текущей реализации оценка для Якоби:

$$\text{ops}_{\text{Jacobi}} \sim \text{iterations} \cdot n^2,$$

так как на каждом шаге выполняется поиск максимального внедиагонального элемента по всей матрице ($O(n^2)$ за шаг поиска).
Для степенного метода оценка считалась через число компонент, число итераций и размерность матрицы.

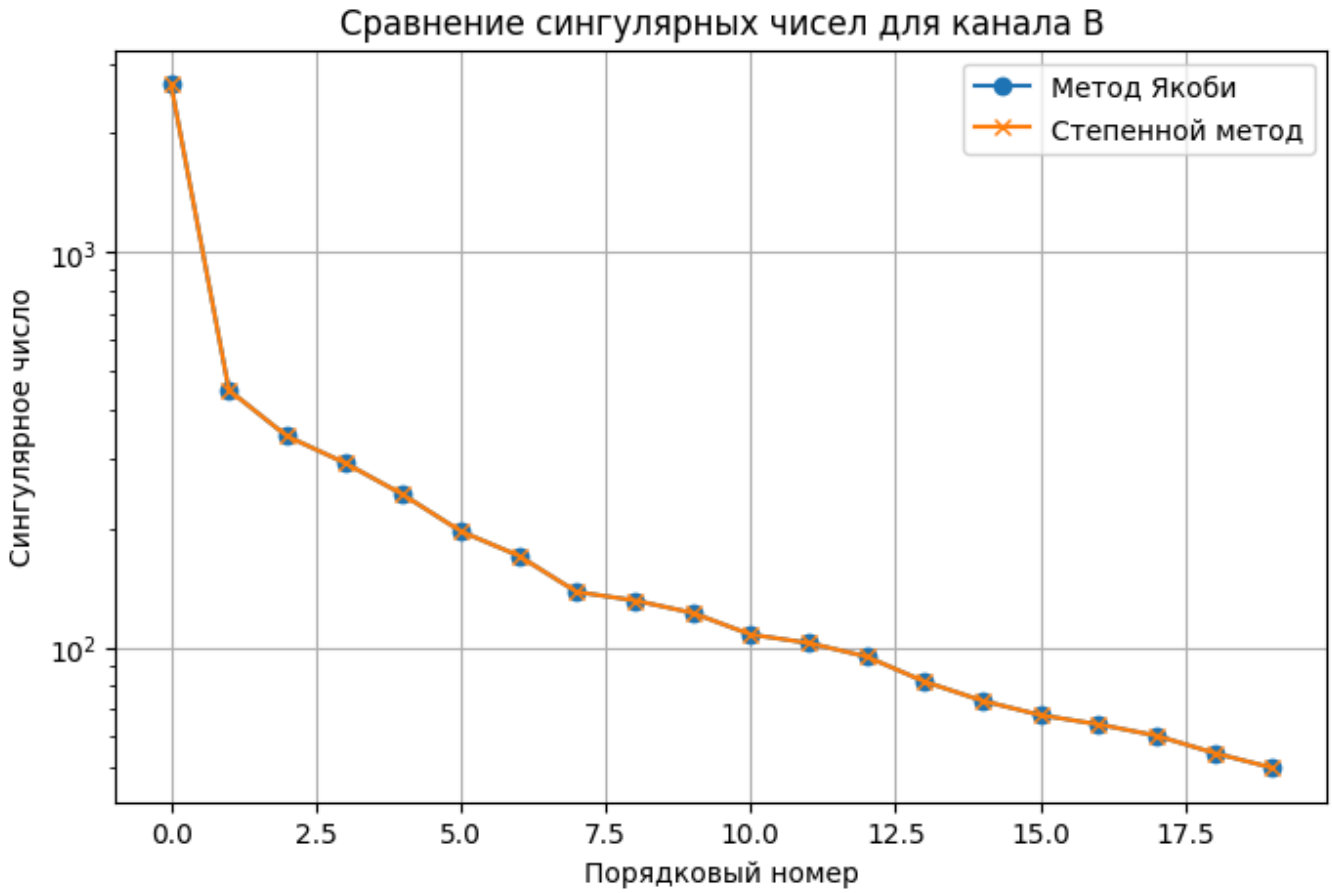


Рисунок 5.4 — Сравнение сингулярных чисел, полученных методом Якоби и степенным методом.

3.6 Листинг кода с подробным пояснением блоков

Блок A. Импорт и служебные функции

Назначение блока: базовая инфраструктура вычислений.

- Подключает библиотеки для линейной алгебры, визуализации, работы с изображениями и замера времени.
- Задает вспомогательные функции норм:
  - `norm_2` — евклидова норма вектора;
  - `absolute_fro_error` — абсолютная ошибка восстановления;
  - `relative_fro_error` — относительная ошибка;
  - `clip_image_array` — ограничение диапазона пикселей до `[0, 255]`.

Почему это важно: эти функции используются во всех следующих этапах и обеспечивают единый способ оценки качества.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from PIL import Image
from pathlib import Path
import time

np.set_printoptions(precision=6, suppress=True)

def norm_2(x):
    return float(np.sqrt(np.sum(np.asarray(x, dtype=np.float64) ** 2)))

def absolute_fro_error(A, A_restored):
    diff = np.asarray(A, dtype=np.float64) - np.asarray(A_restored,
dtype=np.float64)
    return np.linalg.norm(diff.ravel())

def relative_fro_error(A, A_restored):
    diff = np.asarray(A, dtype=np.float64) - np.asarray(A_restored,
dtype=np.float64)
    base = np.asarray(A, dtype=np.float64)
    return np.linalg.norm(diff.ravel()) / np.linalg.norm(base.ravel())

def clip_image_array(A):
    return np.clip(A, 0, 255).astype(np.uint8)
```

Блок B. Степенной метод и SVD через $A^T A$

Назначение блока: вычислить приближенное SVD для канала изображения.

- `power_method_symmetric` ищет доминирующее собственное значение и собственный вектор симметричной матрицы.
- `svd_power` строит последовательность старших сингулярных компонент:
  1. формирует $M = A^T A$,
  2. запускает степенной метод,
  3. восстанавливает сингулярное число $\sigma = \sqrt{\lambda}$,
  4. находит левый сингулярный вектор $u = Av/\sigma$,
  5. выполняет дефляцию $M \leftarrow M - \lambda vv^T$.
- Встроена проверка отрицательных оценок λ (пункт задания 2.c).

Выход блока: матрицы `U`, `S`, `VT` и функция `reconstruct_matrix`, которая строит приближение ранга `k`.

```
def power_method_symmetric(M, iterations, seed):
    rng = np.random.default_rng(seed)
    x = rng.normal(size=M.shape[0])
    x = x / norm_2(x)

    for _ in range(iterations):
        y = M @ x
        y_norm = norm_2(y)
        if y_norm == 0:
            break
        x_new = y / y_norm
        if np.dot(x_new, x) < 0:
            x_new = -x_new
        x = x_new

    eigenvalue = float((x @ M @ x) / (x @ x))
    return eigenvalue, x

def svd_power(A, rank, iterations, seed=123):
    A = np.asarray(A, dtype=np.float64)
    M = A.T @ A
    M_work = M.copy()

    singular_values = []
    right_vectors = []
    left_vectors = []

    for component in range(rank):
        eigenvalue, v = power_method_symmetric(M_work, iterations, seed +
component)

        if eigenvalue < 0:
            print(
                f"Внимание: получено отрицательное собственное значение
{eigenvalue:.6e} "
                f"на компоненте {component}"
            )
            # Теоретически для A^T A собственные значения неотрицательные.
            # Отрицательная оценка — численный артефакт.
            eigenvalue = 0.0

        sigma = float(np.sqrt(eigenvalue))
        if sigma < 1e-10:
            break

        u = A @ v / sigma
```

```
        u_norm = norm_2(u)
        if u_norm < 1e-12:
            break
        u = u / u_norm

        singular_values.append(sigma)
        right_vectors.append(v)
        left_vectors.append(u)

        # Дефляция
        M_work = M_work - eigenvalue * np.outer(v, v)
        M_work = (M_work + M_work.T) / 2

    U = np.column_stack(left_vectors)
    S = np.array(singular_values)
    VT = np.vstack(right_vectors)
    return U, S, VT

def reconstruct_matrix(U, S, VT, k):
    k = min(k, len(S))
    return U[:, :k] @ np.diag(S[:k]) @ VT[:k, :]
```

Блок C. Загрузка изображения и подготовка каналов

Назначение блока: получить входные данные для всех вычислений.

- Загружает изображение, приводит его к `RGB` и фиксированному размеру `640x640`.
- Преобразует изображение в числовой массив `float64`.
- Разделяет изображение на три матрицы каналов `R`, `G`, `B`.

Выход блока: словарь `channels`, который далее передается в расчет SVD.

```
# путь к изображению задается вручную
image_path = Path("input.jpg")

image_original = Image.open(image_path).convert("RGB")
target_size = (640, 640)
image = image_original.resize(target_size, Image.Resampling.LANCZOS)
image_array = np.asarray(image, dtype=np.float64)

R = image_array[:, :, 0]
G = image_array[:, :, 1]
B = image_array[:, :, 2]
channels = {"R": R, "G": G, "B": B}
```

Блок D. Расчет метрик по $N = 10, 50, 100$

Назначение блока: проверить влияние числа итераций степенного метода на результат.

- Для каждого `N` и каждого канала:
  - считает SVD,
  - восстанавливает матрицу канала,
  - считает абсолютную и относительную ошибки,
  - фиксирует время выполнения.
- Результаты складываются в структуру `power_svd_results` и итоговую таблицу `power_metrics_df`.

Что анализируется по таблице: сходимость, устойчивость спектра и рост вычислительных затрат при увеличении `N`.

```
iteration_values = [10, 50, 100]
power_rank = 40

power_svd_results = {}
power_metrics = []

for iterations in iteration_values:
    power_svd_results[iterations] = {}

    for channel_name, channel_matrix in channels.items():
        start_time = time.perf_counter()
        U, S, VT = svd_power(channel_matrix, rank=power_rank,
iterations=iterations, seed=1000 + iterations)
        elapsed_time = time.perf_counter() - start_time

        restored = reconstruct_matrix(U, S, VT, len(S))
        abs_error = absolute_fro_error(channel_matrix, restored)
        rel_error = relative_fro_error(channel_matrix, restored)

        power_svd_results[iterations][channel_name] = {
            "U": U,
            "S": S,
            "VT": VT,
            "restored": restored,
            "abs_error": abs_error,
            "rel_error": rel_error,
            "time": elapsed_time
        }
```

```
        power_metrics.append({
            "Канал": channel_name,
            "N": iterations,
            "Absolute Frobenius error": abs_error,
            "Относительная ошибка восстановления": rel_error,
            "Число найденных сингулярных чисел": len(S),
            "Время, сек": elapsed_time
        })

power_metrics_df = pd.DataFrame(power_metrics)
print(power_metrics_df)
```

Блок E. Усеченное SVD и визуализация качества

Назначение блока: оценить компромисс «качество/сжатие».

- Функция `restore_rgb_from_power_results` собирает RGB-изображение из усеченных компонент каналов.
- Для `k = 5,10,20,40` строятся восстановленные версии.
- Для каждой версии считаются:
  - абсолютная ошибка,
  - относительная ошибка,
  - степень сжатия.

Выход блока: таблица `rank_error_table` для ответа на вопросы анализа результатов.

```
def restore_rgb_from_power_results(results_for_iterations, k):
    restored_channels = []
    for channel_name in ["R", "G", "B"]:
        U = results_for_iterations[channel_name]["U"]
        S = results_for_iterations[channel_name]["S"]
        VT = results_for_iterations[channel_name]["VT"]
        restored_channel = reconstruct_matrix(U, S, VT, k)
        restored_channels.append(restored_channel)
    restored_rgb = np.dstack(restored_channels)
    return clip_image_array(restored_rgb)

rank_values = [5, 10, 20, 40]
restored_images = {}
for k in rank_values:
    restored_images[k] =
restore_rgb_from_power_results(power_svd_results[100], k)

rank_error_rows = []
```

```
for k in rank_values:
    restored_float = restored_images[k].astype(np.float64)
    abs_error = absolute_fro_error(image_array, restored_float)
    rel_error = relative_fro_error(image_array, restored_float)
    compression = (image_array.size) / (k * (image_array.shape[0] +
image_array.shape[1] + 1) * 3)

    rank_error_rows.append({
        "k": k,
        "Absolute Frobenius error": abs_error,
        "Относительная ошибка изображения": rel_error,
        "Степень сжатия": compression
    })

rank_error_table = pd.DataFrame(rank_error_rows)
print(rank_error_table)
```

Блок F. Метод Якоби (полная проблема для уменьшенного канала)

Назначение блока: решить полную проблему собственных значений для выбранного канала и сравнить результат со степенным методом.

- Масштабирует изображение до `64x64` для приемлемого времени расчетов.
- Формирует матрицу $A^T A$ выбранного канала.
- Реализует вращения Якоби:
  - поиск максимального внедиагонального элемента,
  - вычисление параметров вращения,
  - обновление матрицы и накопление собственных векторов.

Выход блока: собственные значения/векторы, число итераций, признак сходимости и история уменьшения внедиагональных элементов.

```
jacobi_size = 64
image_small = image.resize((jacobi_size, jacobi_size),
Image.Resampling.LANCZOS)
image_small_array = np.asarray(image_small, dtype=np.float64)

jacobi_channel_name = "B"
A_jacobi = image_small_array[:, :, 2]
ATA_jacobi = A_jacobi.T @ A_jacobi

def max_offdiag_abs(M):
    M_abs = np.abs(M).copy()
```

```
    np.fill_diagonal(M_abs, 0.0)
    index = np.unravel_index(np.argmax(M_abs), M.shape)
    return float(M_abs[index]), index

def jacobi_rotation_parameters(app, aqq, apq):
    denom = app - aqq
    if abs(denom) < 1e-30:
        t = np.sign(apq) * np.inf if apq != 0 else 0.0
    else:
        t = 2.0 * apq / denom

    if np.isinf(t):
        c = np.sqrt(0.5)
        s = np.sign(t) * np.sqrt(0.5)
    else:
        inv = 1.0 / np.sqrt(1.0 + t * t)
        c = np.sqrt(0.5 * (1.0 + inv))
        s = np.sign(t) * np.sqrt(0.5 * (1.0 - inv))
    return t, c, s

def jacobi_eigen_symmetric(A, eps=1e-5, max_iter=100000):
    A = np.asarray(A, dtype=np.float64)
    n = A.shape[0]
    B = A.copy()
    V = np.eye(n)
    history = []

    for iteration in range(max_iter + 1):
        max_value, (p, q) = max_offdiag_abs(B)
        history.append({"iteration": iteration, "max_offdiag_abs": max_value,
"p": p, "q": q})

        if max_value < eps:
            return {
                "eigenvalues": np.diag(B).copy(),
                "eigenvectors": V.copy(),
                "iterations": iteration,
                "max_offdiag_abs": max_value,
                "converged": True,
                "history": pd.DataFrame(history),
            }

        app = B[p, p]
        aqq = B[q, q]
        apq = B[p, q]
        _, c, s = jacobi_rotation_parameters(app, aqq, apq)
```

```
        for i in range(n):
            if i != p and i != q:
                bip = B[i, p]
                biq = B[i, q]
                B[i, p] = c * bip + s * biq
                B[p, i] = B[i, p]
                B[i, q] = -s * bip + c * biq
                B[q, i] = B[i, q]

        B[p, p] = c * c * app + 2.0 * c * s * apq + s * s * aqq
        B[q, q] = s * s * app - 2.0 * c * s * apq + c * c * aqq
        B[p, q] = 0.0
        B[q, p] = 0.0

        Vp = V[:, p].copy()
        Vq = V[:, q].copy()
        V[:, p] = c * Vp + s * Vq
        V[:, q] = -s * Vp + c * Vq

    return {
        "eigenvalues": np.diag(B).copy(),
        "eigenvectors": V.copy(),
        "iterations": max_iter,
        "max_offdiag_abs": max_offdiag_abs(B)[0],
        "converged": False,
        "history": pd.DataFrame(history),
    }
```

Блок G. Сравнение Якоби и степенного метода

Назначение блока: получить численное сравнение методов на одинаковой задаче.

- Считает сингулярные числа методом Якоби.
- Считает первые `jacobi_compare_rank` сингулярных чисел степенным методом.
- Формирует таблицу `jacobi_power_comparison` с абсолютным отклонением по компонентам.
- Формирует `operation_table` (размерность, параметр, оценка операций, время).

Что показывает блок: точность совпадения спектра и разница в вычислительной стоимости.

```
start_time = time.perf_counter()
jacobi_result = jacobi_eigen_symmetric(ATA_jacobi, eps=1e-5, max_iter=100000)
jacobi_time = time.perf_counter() - start_time
```

```
jacobi_eigenvalues = np.where(jacobi_result["eigenvalues"] > 0,
jacobi_result["eigenvalues"], 0)
jacobi_singular_values = np.sort(np.sqrt(jacobi_eigenvalues))[::-1]

jacobi_compare_rank = 20
start_time = time.perf_counter()
U_power_small, S_power_small, VT_power_small = svd_power(
    A_jacobi, rank=jacobi_compare_rank, iterations=100, seed=777
)
power_small_time = time.perf_counter() - start_time

comparison_count = min(jacobi_compare_rank, len(S_power_small),
len(jacobi_singular_values))
jacobi_power_comparison = pd.DataFrame({
    "Номер": np.arange(1, comparison_count + 1),
    "Степенной метод": S_power_small[:comparison_count],
    "Метод Якоби": jacobi_singular_values[:comparison_count],
    "Абсолютное отклонение": np.abs(S_power_small[:comparison_count] -
jacobi_singular_values[:comparison_count]),
})

operation_table = pd.DataFrame({
    "Метод": ["Степенной метод", "Метод вращений Якоби"],
    "Размерность матрицы сравнения": [A_jacobi.shape, ATA_jacobi.shape],
    "Основной параметр": [f"{jacobi_compare_rank} сингулярных чисел, 100
итераций", f"{jacobi_result['iterations']} вращений"],
    "Оценка числа базовых операций": [jacobi_compare_rank * 100 *
ATA_jacobi.shape[0] ** 2, jacobi_result["iterations"] * ATA_jacobi.shape[0] **
2],
    "Время, сек": [power_small_time, jacobi_time],
})

print(jacobi_power_comparison)
print(operation_table)
```

Блок H. Гистограммы (творческое задание 5.a и 5.b)

Назначение блока: проанализировать распределение яркости до и после усечения SVD.

- Строит отдельные гистограммы исходных каналов `R/G/B`.
- Строит сравнительные гистограммы «исходный канал vs восстановленный канал» при выбранном `k`.

Что видно по графикам: сглаживание распределения и уменьшение вклада высокочастотных колебаний при сохранении основной формы распределения.

```
# Гистограммы исходных каналов
for channel_name, channel_matrix in channels.items():
    plt.figure(figsize=(6, 4))
    plt.hist(channel_matrix.ravel(), bins=256, range=(0, 255))
    plt.title(f"Гистограмма яркости канала {channel_name}")
    plt.xlabel("Яркость")
    plt.ylabel("Частота")
    plt.grid(alpha=0.2)
    plt.show()

# Сравнение гистограмм исходного и восстановленного изображения (например,
k=40)
selected_k = 40
restored_selected = restored_images[selected_k].astype(np.float64)
restored_channels = {
    "R": restored_selected[:, :, 0],
    "G": restored_selected[:, :, 1],
    "B": restored_selected[:, :, 2],
}

for channel_name in ["R", "G", "B"]:
    plt.figure(figsize=(7, 4))
    plt.hist(channels[channel_name].ravel(), bins=256, range=(0, 255),
alpha=0.5, label="Исходный канал")
    plt.hist(restored_channels[channel_name].ravel(), bins=256, range=(0,
255), alpha=0.5, label=f"Восстановление k={selected_k}")
    plt.title(f"Сравнение гистограмм канала {channel_name}")
    plt.xlabel("Яркость")
    plt.ylabel("Частота")
    plt.legend()
    plt.grid(alpha=0.2)
    plt.show()
```

---

4. Анализ результатов (по вопросам задания)

4.1 Как число итераций степенного метода влияет на точность разложения?

При увеличении числа итераций N улучшается сходимость собственных векторов и стабилизируются оценки старших сингулярных чисел. На практике переход от $N = 10$ к $N = 50$ дает заметное улучшение, а $N = 100$ — наиболее устойчивые результаты из рассмотренных.
Также при малом N чаще проявляются численные артефакты (в том числе отрицательные оценки после дефляции).

4.2 Какое минимальное число сингулярных чисел обеспечивает приемлемое качество изображения?

По визуальному сравнению восстановлений:

- при $k = 5$ изображение чрезмерно грубое;
- при $k = 10$ сохраняются основные крупные формы, но деталей мало;
- при $k \approx 20$ качество становится визуально приемлемым;
- при $k = 40$ качество заметно стабильнее и ближе к исходному.

Итог: минимально приемлемое k — около 20, комфортное качество — 40.

4.3 Какой метод (степенной или Якоби) оказался более эффективным по числу операций?

В проведенном эксперименте степенной метод оказался выгоднее по вычислительной стоимости для практической задачи сжатия, так как целевыми являются только старшие сингулярные компоненты.
Метод Якоби решает полную проблему собственных значений и дает более универсальный результат, но требует больше операций и времени.

4.4 Есть ли различия в распределении сингулярных чисел для разных цветовых каналов?

Да, различия есть: каналы R, G, B имеют разную структуру яркости и контрастов, поэтому у них различается скорость убывания сингулярных чисел и доля энергии, сосредоточенная в старших компонентах.
Это видно по разным кривым на графиках сингулярного спектра для каналов.

4.5 Какова степень сжатия данных при использовании усеченного SVD?

Для одного канала вместо хранения $n \cdot m$ значений хранится:

$$k(n + m + 1).$$

Для RGB-изображения (три канала) объем масштабируется втрое.
Следовательно, степень сжатия растет при уменьшении k, но одновременно растет ошибка восстановления. Это подтверждено таблицей зависимости ошибки и степени сжатия от k.

---

5. Творческое задание (дополнительно)

5.a Гистограммы распределения значений пикселей по каналам

Для каждого канала R, G, B построены гистограммы (`np.histogram` / `plt.hist`) по диапазону яркости $[0, 255]$.
Гистограмма показывает, как распределены интенсивности пикселей, и позволяет оценить, какие уровни яркости преобладают в каждом канале.

5.b Как SVD влияет на распределение яркости

После восстановления усеченным SVD гистограммы становятся более сглаженными:

- уменьшается вклад редких локальных значений (высокочастотные детали/шум),
- основная масса распределения смещается к более «усредненным» уровням,
- контуры распределения исходного и восстановленного каналов остаются близкими при достаточном k (например, $k = 40$).

Таким образом, усеченное SVD частично фильтрует шум и мелкие колебания яркости, сохраняя основную структуру распределения.

Распределение яркости исходных каналов изображения.

Поканально (до восстановления):

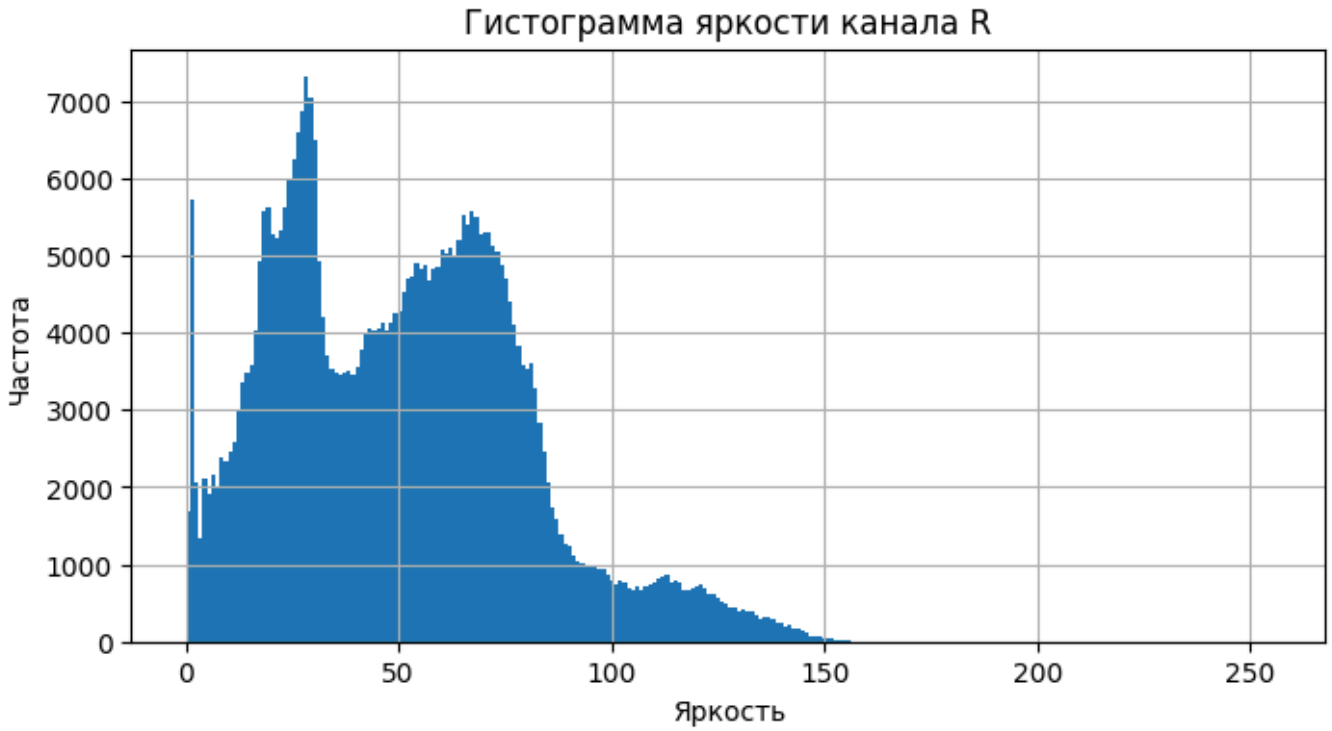


Рисунок 6.1 — Гистограмма яркости канала R до восстановления.

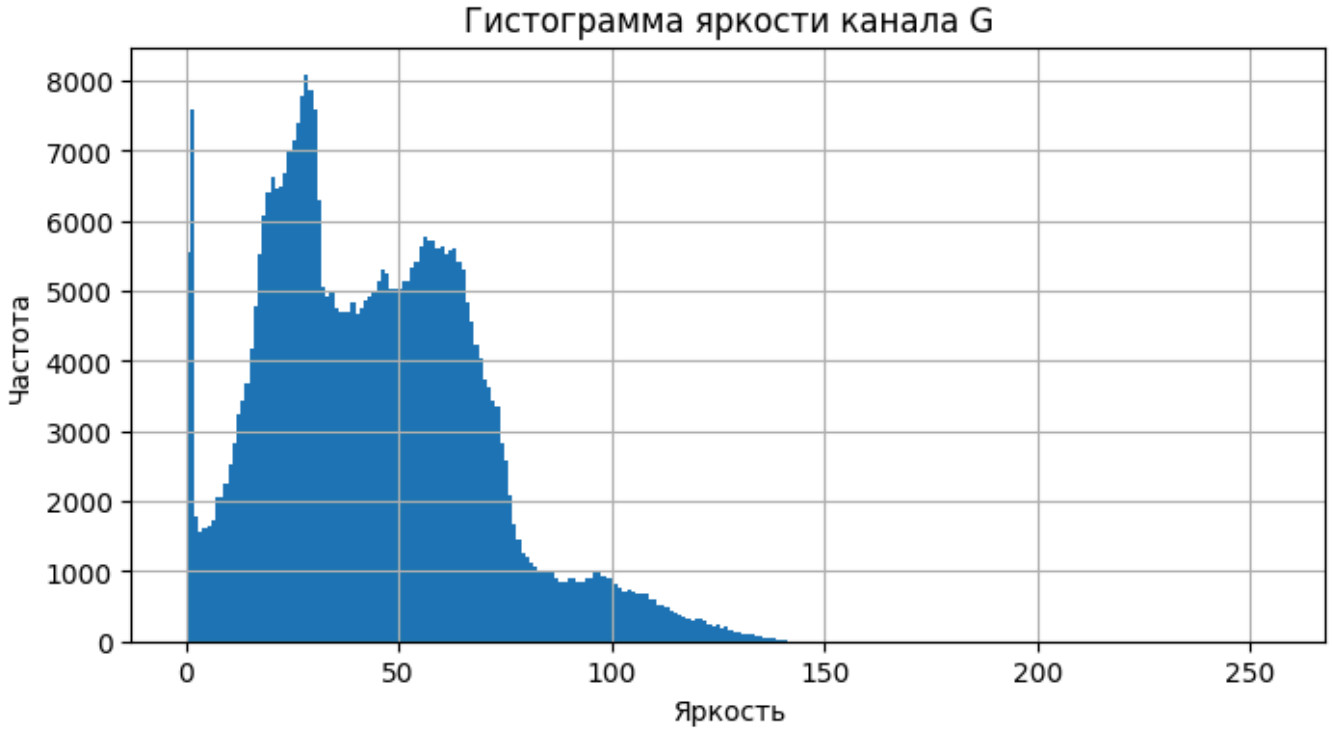


Рисунок 6.2 — Гистограмма яркости канала G до восстановления.

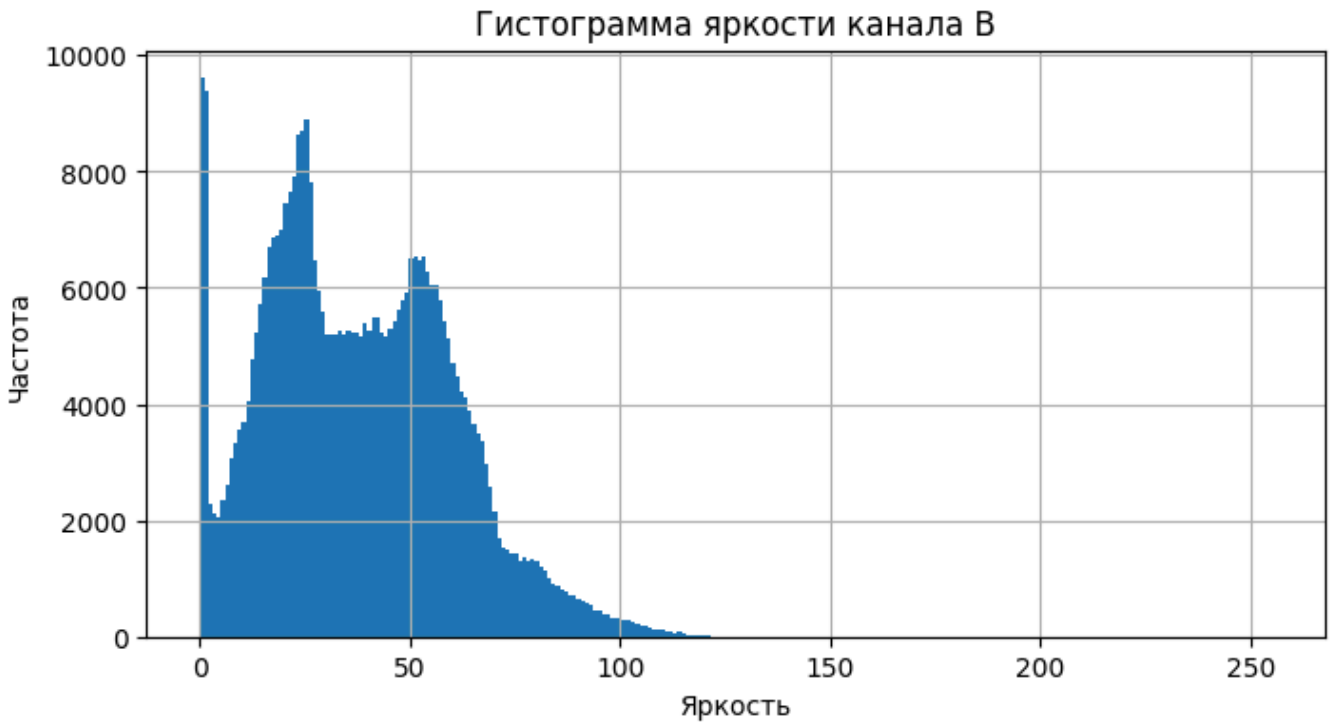


Рисунок 6.3 — Гистограмма яркости канала B до восстановления.

Сравнение распределения яркости до и после восстановления усеченным SVD.

Поканально (сравнение до/после восстановления):

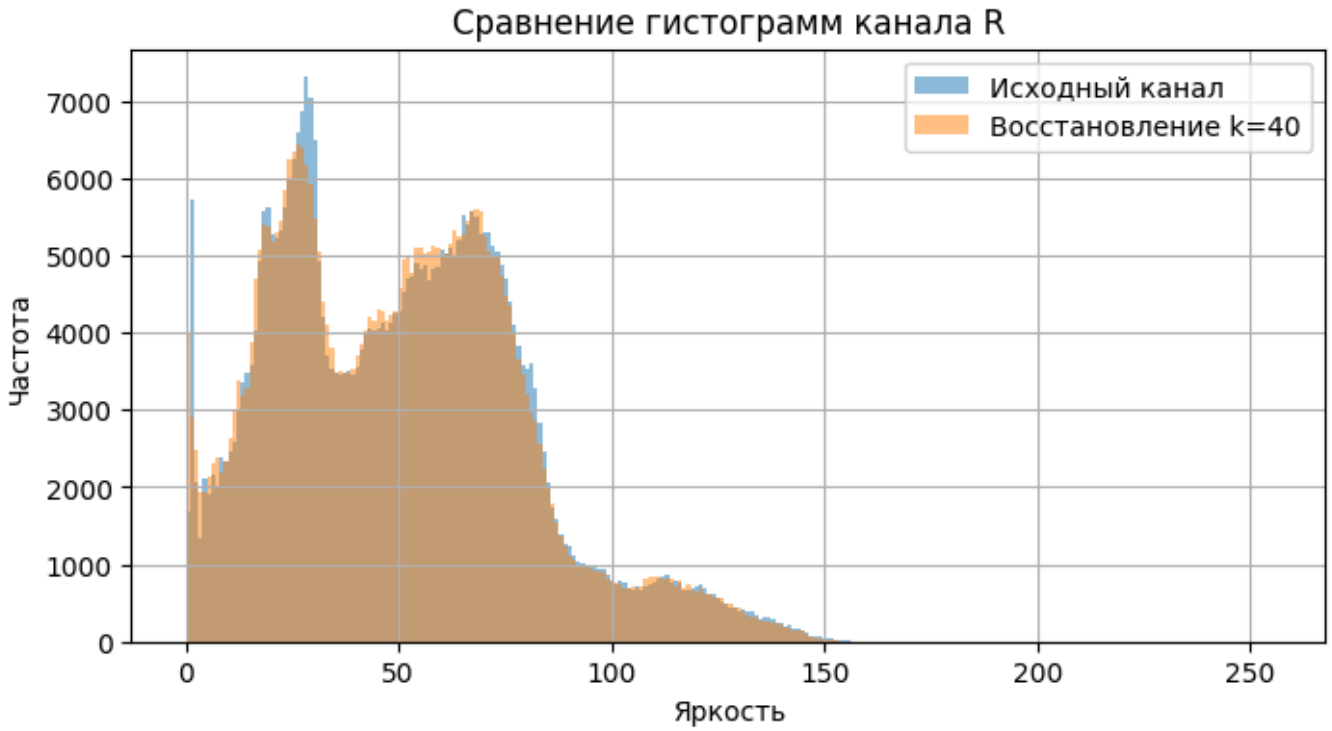


Рисунок 7.1 — Сравнение гистограмм яркости канала R до и после восстановления.

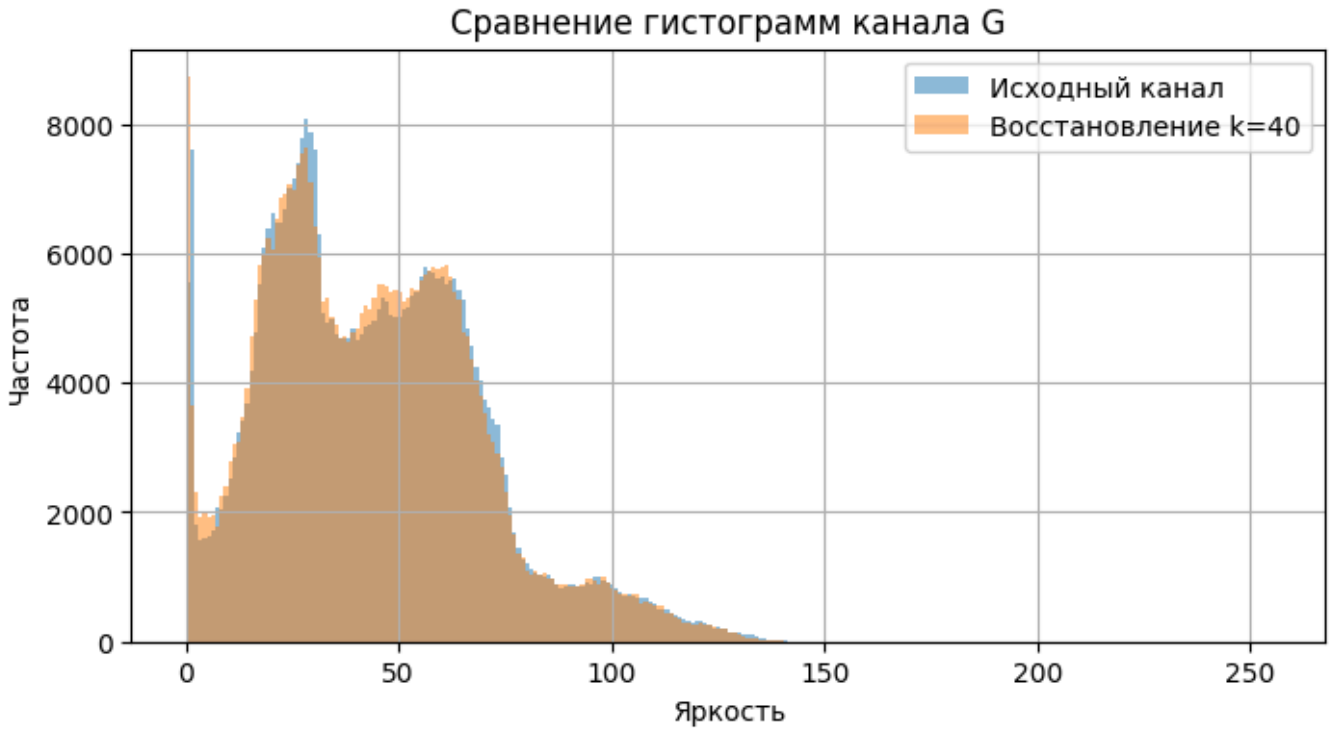


Рисунок 7.2 — Сравнение гистограмм яркости канала G до и после восстановления.

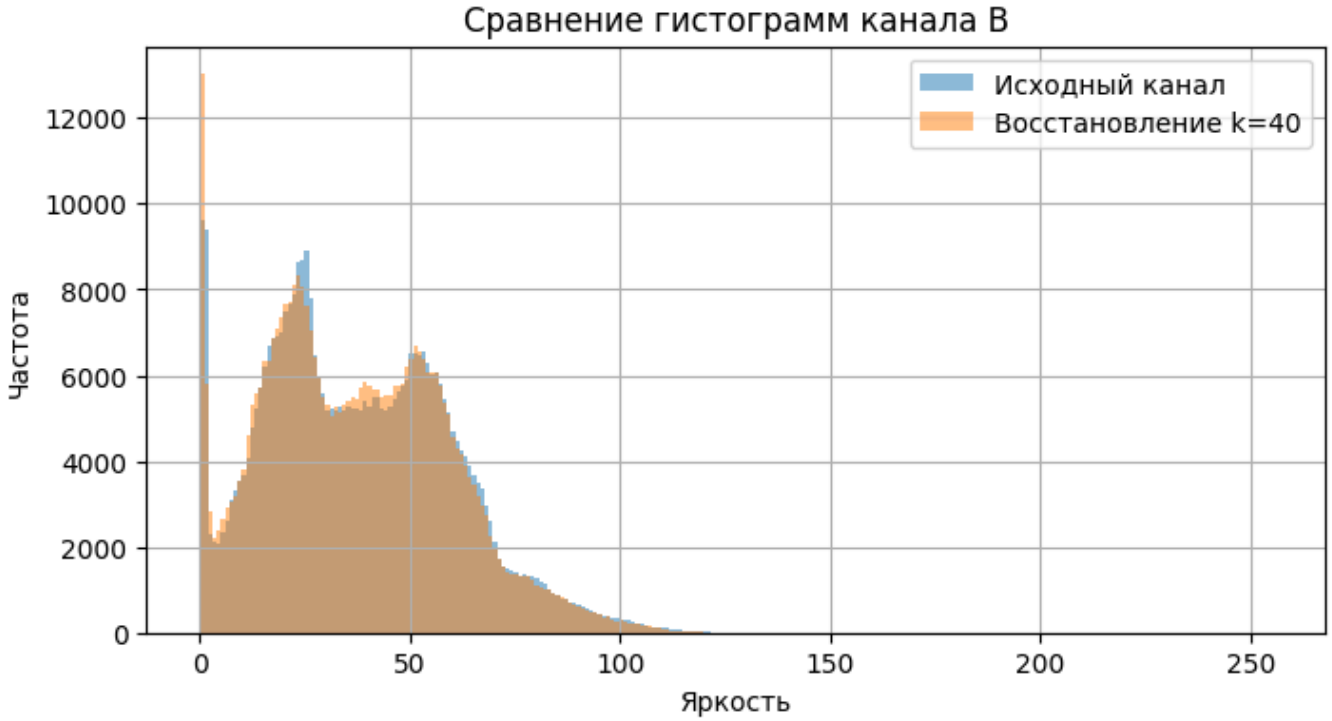


Рисунок 7.3 — Сравнение гистограмм яркости канала B до и после восстановления.

5.c Свой вариант применения SVD

Возможные практические применения:

1. Удаление шума с фото (отсечение младших сингулярных компонент).
2. Выделение признаков лиц в задачах распознавания (Eigenfaces).

3. Сжатие видеоданных и матриц межкадровой разницы.

---

6. Выводы

В работе реализованы и сравнены два подхода:

- степенной метод для получения старших сингулярных компонент;
- метод Якоби для полной проблемы собственных значений.

Показано, что усеченное SVD эффективно для сжатия изображения и позволяет управлять компромиссом между качеством и объемом данных через параметр k.
Для прикладной задачи сжатия более эффективен степенной метод, а метод Якоби полезен как точный ориентир при решении полной проблемы собственных значений на матрицах умеренной размерности.